ASL Smart Translator

1 Problem and Motivation

According to the World Federation of the Deaf, there are more than 70 million deaf people worldwide. And yet, according to a study by the 2010–2018 National Health Interview, it was found that in the USA, sign language use among adults (including those with and without hearing loss) was roughly 2.80%.

Let us zoom into India itself. According to the 2011 Census, the total population of deaf persons in India numbered about 50 lakh. Yet, as reported by the India Sign Language Research and Training Center, the needs of these people were severely ignored. The teaching methods needed urgent attention, while ISL existed, a lot of parents weren't aware of its ability to ease communication barriers. At that time, there were only 300 certified interpreters despite the urgent requirement in various fields.

All of this shows us how communication becomes a significant barrier for individuals who rely on sign language on a day-to-day basis. This is especially felt in fields such as healthcare and education. While alternatives such as text-based communication and interpreters exist, they are not practical or sometimes not available for real-time situations.

Recent developments in AI and Machine Learning are helping improve communication and making the world more accessible for people with disabilities. Examples include the RayBan Meta Smart Glasses that help the blind community, or Google's upcoming SignGemma. Similarly, to bridge this gap, we wanted to make a project that can help recognize ASL and translate it for individuals with the help of generative AI, LLM models, and machine learning models.

2 Method

The proposed system combines the concepts of machine learning, natural language processing, and computer vision.

The system captures hand gestures using a webcam. It then extracts the hand landmarks using MediaPipe and predicts which gesture it is. This is done using a trained machine learning model. It then converts the output into a proper sentence and translates it into Hindi.

The overall pipeline is shown in the figure below.

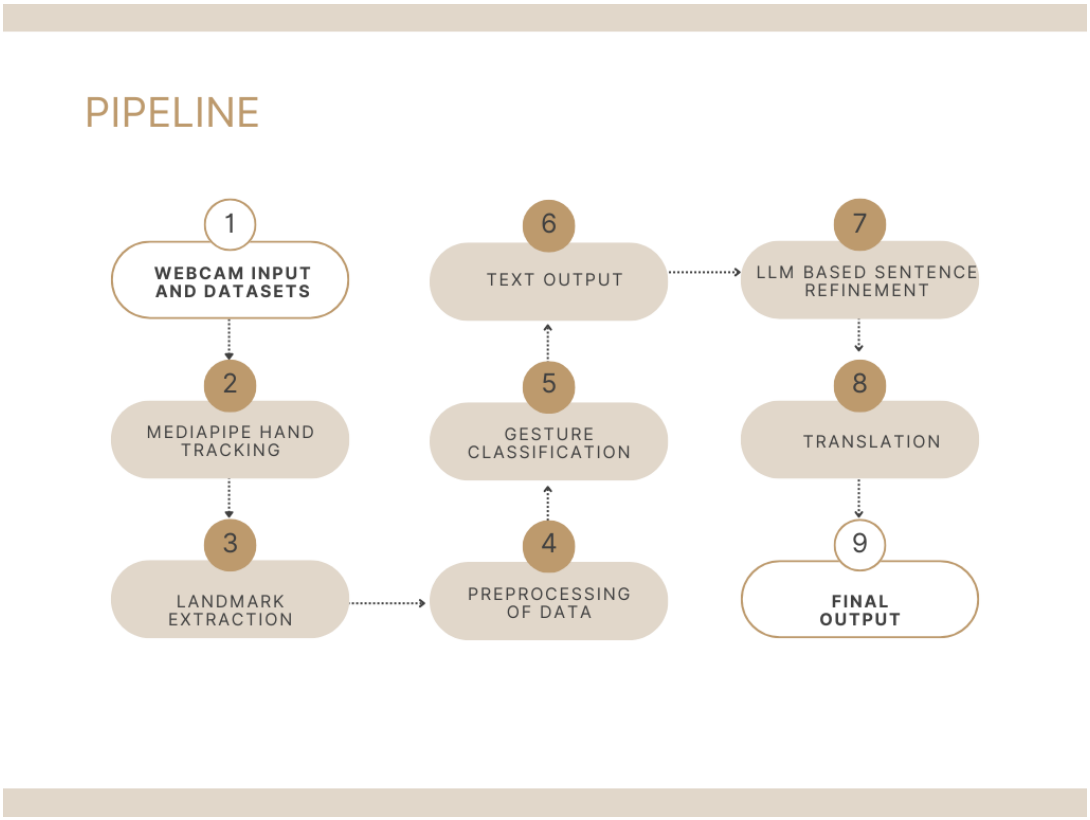

Figure 1: Overall Pipeline of the System

The dataset was collected in two ways:

1. First, we used the dataset available on Kaggle called WS_ASL. This contained the entire ASL dataset.

2. We also recorded our own inputs for certain key phrases, alphabets, and numbers.

The videos collected were recorded in different lighting conditions, different hand positions, and by different people to make the model more fit for real-time situations. This was also done to ensure that the model does not learn a bias for a specific person and that there is sufficient variation.

Then, we took the CSV values of both datasets. We obtained the CSV values of the dataset we recorded using MediaPipe. MediaPipe collected 21 landmarks for each hand and each gesture. These landmarks were mapped in (x, y, z) coordinates.

To improve consistency, normalization was applied to the landmarks. This reduced the effect of hand position and camera distance during training and inference. It was done relative to a base landmark point on the hand.

These landmarks were then used to train a machine learning classifier which would help us in gesture recognition. We first used Random Forest to train our dataset. While Random Forest worked well for phrases and numbers, it did not predict gestures with high accuracy for alphabets. To combat this issue, we switched to XGBoost. When XGBoost showed better results overall, we updated our code and continued with XGBoost.

During real-time inference, the webcam frames are continuously processed using the same landmark extraction and pre-processing pipeline used during data training. The trained model predicts the corresponding ASL alphabet, gesture, or number.

After prediction, sentence building and smoothing are done using LLMs to ensure the gestures form a valid and understandable sentence. The output is also translated into Hindi to make it more accessible for a larger population.

3 Experiments and Results

3.1 Experimental Setup

The system was evaluated using landmarks generated from the ASL dataset containing alphabets, numbers, and phrases. The dataset was a combination of CSV values recorded by us using MediaPipe, as well as CSV values from the dataset found online on Kaggle.

The dataset was split into training and testing sets. Two machine learning approaches were tested: Random Forest and XGBoost. Model performance was evaluated using accuracy, precision, recall, F1-score, and confusion matrices.

3.2 Random Forest Results

Initially, Random Forest was used as the primary classifier for gestures. The model showed success for phrase and number recognition.

The table below shows the accuracy of the model. While there is high accuracy for numbers, there is lower performance for phrases due to increased gesture complexity.

Dataset	**Accuracy**	**Macro Avg Precision**	**Macro Avg Recall**	**Macro Avg F1 Score**	**Weighted Avg F1 Score**
Numbers	99.21%	0.99	0.99	0.99	0.99
Alphabets	89.66%	0.92	0.89	0.88	0.89
Phrases	74.25%	0.83	0.73	0.75	0.76

Table 1: Random Forest Results

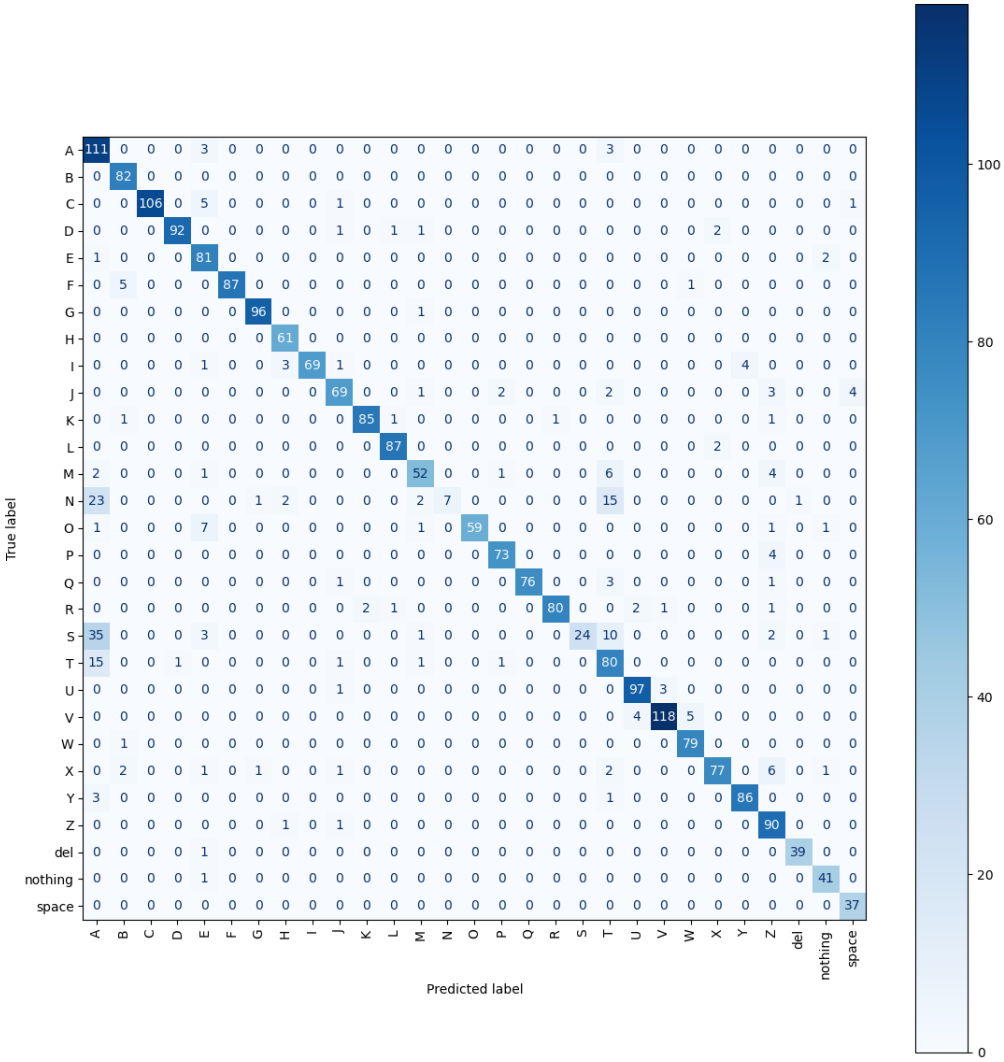

Figure 2: Alphabet Landmarks Confusion Matrix

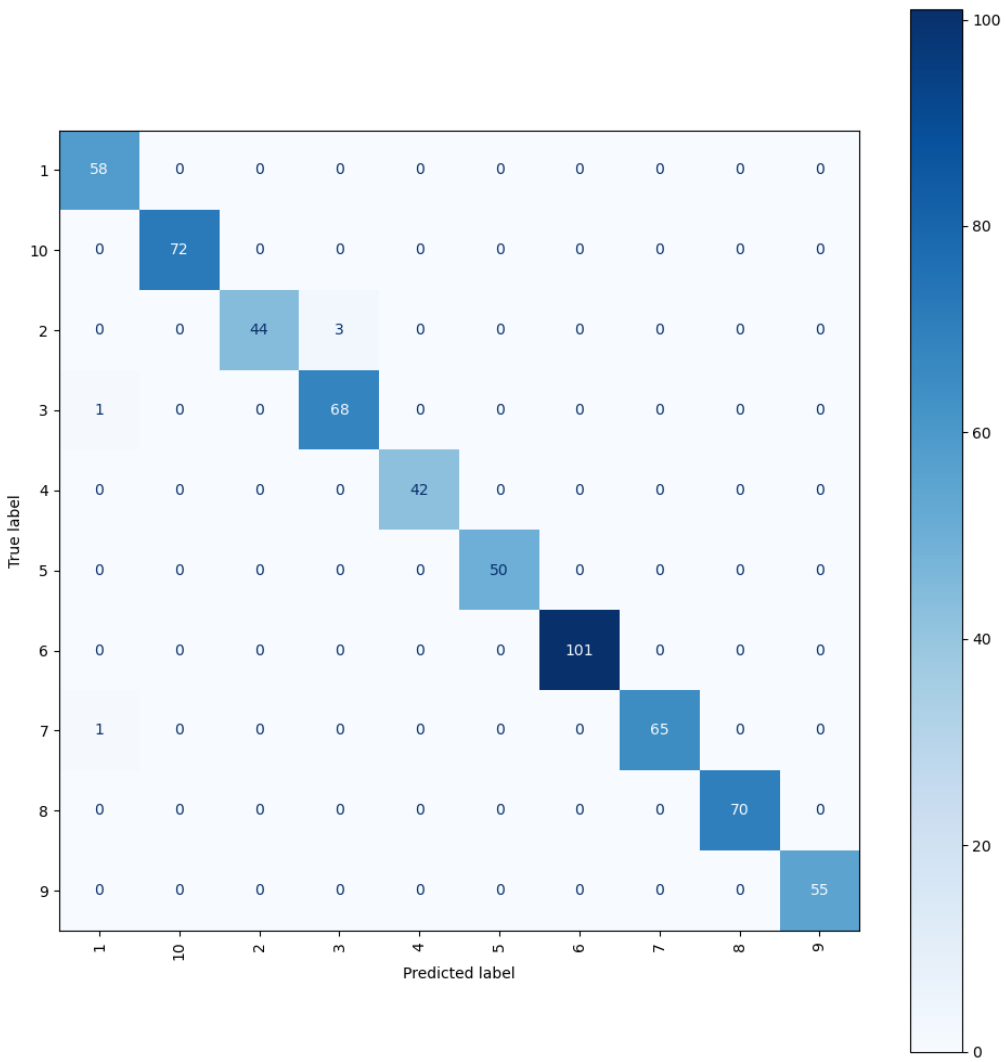

Figure 3: Number Landmarks Confusion Matrix

3.3 XGBoost Results

To improve prediction accuracy, XGBoost was later implemented as the primary classifier. Compared to Random Forest, XGBoost produced significantly more stable and accurate predictions across all datasets.

The XGBoost model achieved substantial improvements in both alphabet and phrase recognition tasks. The confusion matrices showed significantly fewer misclassifications compared to the Random Forest model. The use of x, y, and z landmark coordinates also improved recognition by capturing depth-based spatial information.

Dataset	**Accuracy**	**Macro Avg Precision**	**Macro Avg Recall**	**Macro Avg F1 Score**	**Weighted Avg F1 Score**
Alphabets	97.28%	0.97	0.97	0.97	0.97
Numbers	99.84%	1.00	1.00	1.00	1.00
Phrases	95.84%	0.96	0.96	0.96	0.96

Table 2: XGBoost Results

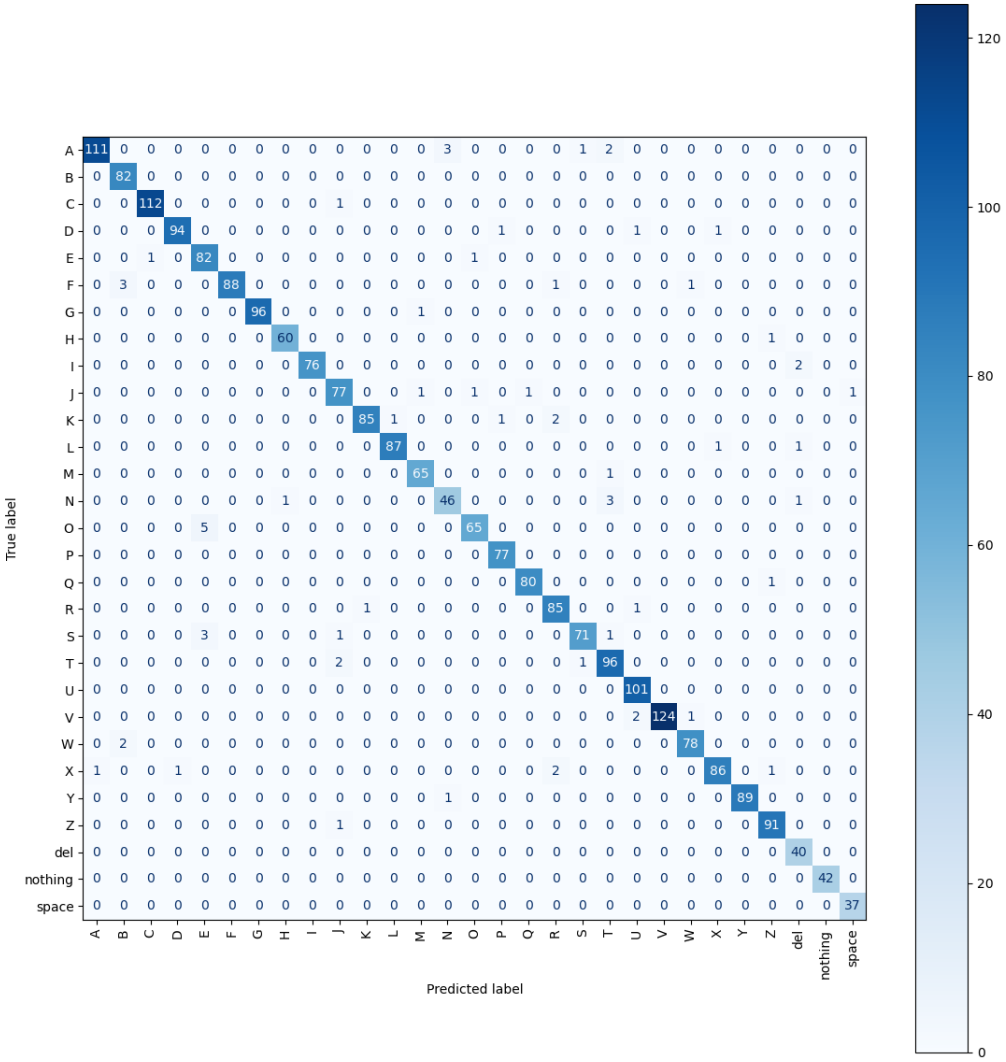

Figure 4: Alphabet Landmarks Confusion Matrix

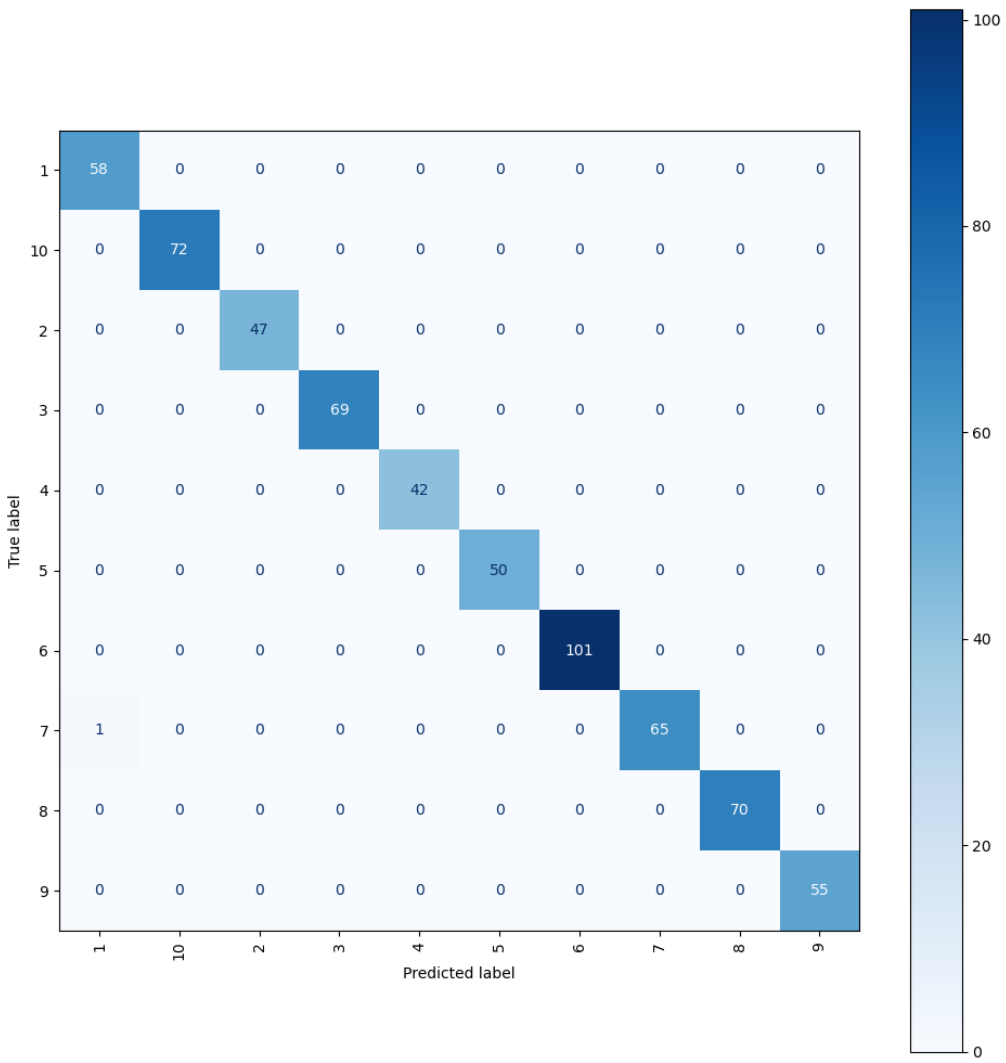

Figure 5: Number Landmarks Confusion Matrix

3.4 Model Comparison and Ablation

A comparison between Random Forest and XGBoost was performed to evaluate the effect of model selection on recognition performance.

Task	**Random Forest**	**XGBoost**
Alphabet Recognition	89.66%	97.28%
Phrases Recognition	74.25%	95.84%

Table 3: Comparison Between Random Forest and XGBoost

4 Analysis

One major strength of the project was using MediaPipe for real-time hand landmarks instead of using raw image frames. This allowed the spatial hand information to be preserved while reducing computational complexity.

The system performed reasonably well for number and phrase recognition during real-time testing. The text output and sentence refinement pipeline also made it easier to understand context even if complete sentences were not being signed.

During development, we used multiple approaches for gesture recognition. Initially, we used CNN-based image classification, but it did not provide accurate predictions because it was not tuned to real-time settings such as variations in lighting and hand orientations.

We then shifted to a landmark-based classification approach by applying Random Forest. This significantly improved prediction performance, especially for phrases and numbers. However, alphabet recognition accuracy was still not satisfactory.

To further improve performance, we implemented XGBoost as the primary classifier. Compared to Random Forest, this handled data more effectively and gave better predictions during testing.

While there has been considerable progress, the system still has improvements to make. Currently, the model is able to translate and understand our ASL signs effectively. However, since none of the members of the team are fluent in ASL, the signing speed may vary for fluent signers. We are yet to determine whether the system would be able to capture dynamic and continuous sentence-level ASL translation with the same accuracy.

Further, real-time prediction can still be affected by lighting conditions and camera quality.

Overall, the project demonstrates how combining computer vision, machine learning, and LLM-based text processing can help reduce communication barriers for sign language users in real-time situations. While the dataset is currently more focused, it provides a practical foundation for more advanced sign language translation systems in the future.

5 Contributions

1. **Shrivatsh K** – Collecting dataset, preprocessing the data, training and testing the model using Random Forest and XGBoost.
2. **Chirag Goyal** – Collecting datasets for alphabets and numbers using MediaPipe and converting them into CSV format.
3. **Meesha Maheshwari** – Using LLMs to refine sentences and translate text, accessing LLM APIs, and developing the UI/UX.
4. **Vinay** – Making the presentation and helping in collecting phrases for the dataset.
5. **Manohar** – Making the presentation and helping in collecting phrases for the dataset.

6 Bibliography

1. `https://islrtc.nic.in/about-us/`
2. `https://pubmed.ncbi.nlm.nih.gov/36423340/`